


جزوهٔ آموزشی مهندسی الکترونیک | ELECTRONICS GUIDE

# راهنمای جامع تایمرها
# در کنترل موتورهای BLDC و PMSM
# با میکروکنترلرهای CH32

از انتخاب تایمر مناسب برای هر موتور و تولید PWM مکمل با زمان مرده، تا سازوکار ترمز و پیش‌بارگذاری، تحریک نمونه‌برداری ADC با TRGO، رابط انکودر، هم‌کاری DMA و مدیریت وقفه‌ها در کنترل FOC

در این جزوه، ترجمهٔ فارسی سند رسمی AN11002 (نسخهٔ ۱٫۰، منتشرهٔ شرکت WCH) را می‌خوانید: تایمرهای پیشرفته (ADTM)، همگانی‌منظوره (GPTM) و ساده‌شده در میکروکنترلرهای سری CH32 با هستهٔ RISC-V برای راندن موتورهای BLDC و PMSM نقش‌های مشخصی دارند؛ پیکربندی حالت‌های PWM هم‌ترازِ لبه و مرکزی، خروجی مکمل و زمان مرده، حفاظت ترمز، تحریک ADC، انکودر، DMA و اولویت‌بندی وقفه‌ها همراه با نمونه‌کدهای واقعی گام‌به‌گام مرور می‌شود.

میکروکنترلرهای CH32 | ADTM و GPTM | PWM مکمل و زمان مرده | کارکرد ترمز و حفاظت موتور

تحریک ADC با TRGO | رابط انکودر افزایشی | هم‌کاری DMA | مدیریت وقفه‌های تایمر

ترجمهٔ فارسی AN11002 V1.0 · **نسخهٔ ۱٫۰** — آبان ۱۴۰۵

فهرست مطالب — برای پرش به بخش موردنظر، روی ردیف آن کلیک کنید؛ شمارهٔ صفحه‌ها بر اساس صفحات فیزیکی PDF است.

# فهرست مطالب


۱ توضیحات و دامنهٔ کاربرد ..... ۳

۲ بخش ۱ | کنترل موتور ..... ۳

1.1 کنترل موتور ..... ۳

1.2 منابع تایمر و انتخاب در کنترل موتور ..... ۵

۳ بخش ۲ | تایمرها در کاربرد موتور ..... ۶

2.1 پیکربندی حالت PWM ..... ۶

2.2 خروجی مکمل و پیکربندی زمان مرده ..... ۸

2.3 پیکربندی کارکرد ترمز ..... ۹

2.4 سازوکار پیش‌بارگذاری ..... ۱۰

2.5 تحریک ADC با تایمر ..... ۱۰

2.6 پیکربندی انکودر ..... ۱۰

2.7 تحریک DMA با TIM ..... ۱۲

2.8 مدیریت و بهینه‌سازی وقفه‌های تایمر ..... ۱۳

۴ تاریخچهٔ نسخه‌ها و بیانیه ..... ۱۴

۵ واژگان کلیدی (واژه‌نامهٔ فنی) ..... ۱۴


### فهرست شکل‌ها، جدول‌ها و کدها


شکل ۱-۱ مدار پل H ..... ۴

شکل ۱-۲ مدار اینورتر سه‌فاز ..... ۵

جدول ۱ دامنهٔ کاربرد سند ..... ۳



**چکیده —** سری CH32 شرکت WCH میکروکنترلرهایی مبتنی بر هستهٔ RISC-V چینگ‌که است که منابع تایمری غنی و قدرتمندی — شامل تایمر پیشرفته (ADTM)، تایمر همگانی‌منظوره (GPTM) و تایمر ساده‌شده — را در خود جای داده و به‌طور گسترده در کنترل FOC موتورهای BLDC (جریان مستقیم بدون جاروبک) و PMSM (سنکرون با مغناطیس دائم) به کار می‌رود. این سند نکات کلیدی بهره‌گیری از تایمرها در کاربردهای موتور را نظام‌مند مرور می‌کند؛ تولید PWM مکمل، پیکربندی زمان مرده، سازوکار حفاظت ترمز، تحریک نمونه‌برداری ADC، رابط انکودر، هم‌کاری DMA و مدیریت وقفه‌ها را در بر می‌گیرد.

مقدمه

# توضیحات و دامنهٔ کاربرد

میکروکنترلرهای سری CH32 شرکت ریزالکترونیک وه‌چی‌انگ (WCH) که بر هستهٔ RISC-V چینگ‌که (Qingke) ساخته می‌شوند، منابع تایمریِ غنی و قدرتمندی را در خود جای داده‌اند؛ از جمله تایمرهای پیشرفته (ADTM)، تایمرهای همگانی‌منظوره (GPTM) و تایمرهای ساده‌شده. این منابع به‌طور گسترده در کنترل FOC موتورهای BLDC (جریان مستقیم بدون جاروبک) و PMSM (سنکرون با مغناطیس دائم) به کار می‌روند. این یادداشت کاربردی، نکات کلیدی بهره‌گیری از تایمرها در کاربردهای موتور روی تراشه‌های WCH را به‌صورت نظام‌مند مرور می‌کند و تولید PWM مکمل، پیکربندی زمان مرده، سازوکار حفاظت ترمز، تحریک نمونه‌برداری ADC، رابط انکودر، هم‌کاری DMA و مدیریت وقفه‌ها را در بر می‌گیرد.

جدول ۱ دامنهٔ کاربرد سند

| دامنهٔ کاربرد | سری |
| --- | --- |
| میکروکنترلرهای همگانی‌منظوره | CH32M007، CH32M030، CH32V203، CH32V307 و مانند این‌ها |

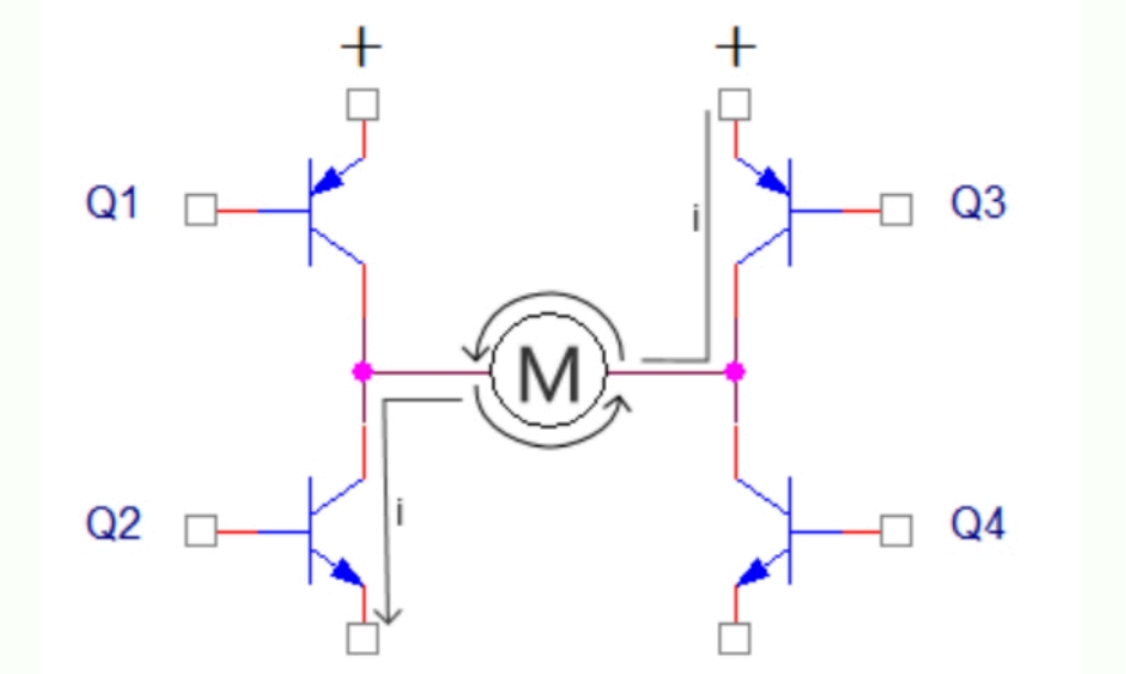
بخش ۱

# کنترل موتور

## ◆ ۱.۱ کنترل موتور

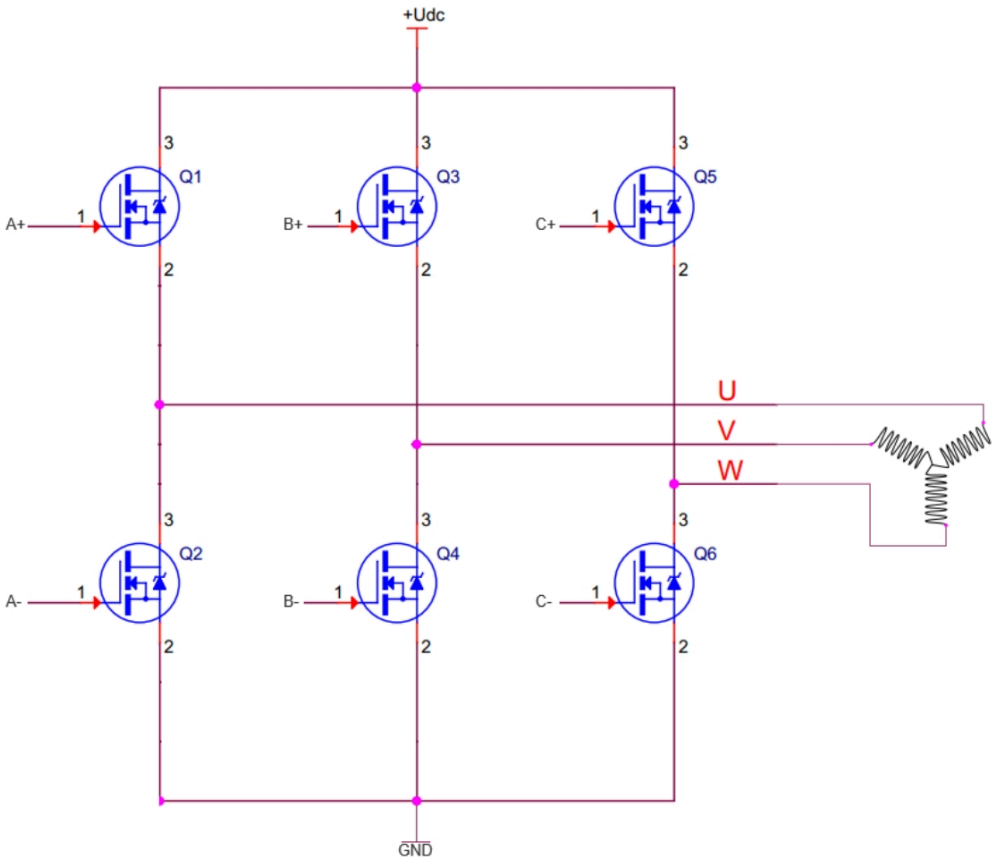
میکروکنترلرهای WCH می‌توانند کنترل سرعت یا موقعیتِ موتورهای جریان مستقیم جاروبک‌دار، موتورهای جریان مستقیم بدون جاروبک (BLDC) و موتورهای سنکرون با مغناطیس دائم (PMSM) را پشتیبانی کنند. نیاز موتورهای گوناگون به کارکردهای تایمر تفاوت چشمگیری دارد.

تنظیم سرعت موتور جریان مستقیم جاروبک‌دار تنها به یک کانال PWM نیاز دارد و جهت چرخش با GPIO معمولی و پل H کنترل می‌شود. چند کانال یک تایمر می‌تواند هم‌زمان چند موتور را برانگیزد. تغییر نسبت وظیفهٔ PWM ولتاژ دو سرِ موتور را عوض می‌کند و بدین‌ترتیب تنظیم سرعت حاصل می‌شود.




شکل ۱-۱ مدار پل H

کنترل شش‌گامِ موج مربعیِ موتور BLDC به سه کانال سیگنال PWM برای راندن پل اینورتر سه‌فاز و همراهی حسگرهای هال برای جابه‌جایی (کموتاسیون) نیاز دارد. چون میکروکنترلر نمی‌تواند ماسفت‌ها را مستقیماً برانگیزد، باید یک تراشهٔ درایور اختصاصی پیکربندی شود. خروجی مکمل و درج زمان مرده در تایمر پیشرفته همین‌جا نقش‌آفرین می‌شوند — با آن‌که در موج مربعیِ شش‌گام در هر لحظه تنها دو فاز رسانا هستند، خروجی مکمل همچنان کلیدِ جلوگیری از اتصال مستقیم بازوهای بالا و پایینِ یک بازو است.




شکل ۱-۲ مدار اینورتر سه‌فاز

کنترل FOC موتور PMSM بیشترین الزام را به منابع تایمر دارد. تایمر باید سه کانال PWM مکملِ هم‌ترازِ مرکزی تولید کند، رویداد به‌روزرسانی، نمونه‌برداری جریان ADC را تحریک کند و سیگنال انکودر یا هال با تایمر همگانی‌منظوره گرفته شود. FOC بی‌حسگر بر این پایه، محاسبات رؤیت‌گر را نیز اضافه می‌کند، اما پیکربندی تایمر با FOC حسگردار تقریباً یکسان است؛ تنها بر دقتِ زمانِ تحریک ADC بیشتر تأکید می‌شود.

## ◆ ۱.۲ منابع تایمر و انتخاب در کنترل موتور

تایمرها به چهار دسته تقسیم می‌شوند: تایمر پیشرفته، تایمر همگانی‌منظوره، تایمر ساده‌شده/پایه و تایمر زمان‌بندِ سیستم. در کنترل موتور، تقسیم نقش میان آن‌ها روشن و مشخص است.

**تایمر پیشرفته (ADTM)** واحد اصلی راندن مرحلهٔ توان است. برای نمونه در CH32V203، تایمر پیشرفته شمارندهٔ 16 بیتیِ خودبارگذارِ صعودی/نزولی است که افزون بر کارکردهای کامل تایمر همگانی‌منظوره، ویژگی‌های اختصاصیِ موتوری نیز دارد: خروجی PWM مکمل با زمان مردهٔ قابل برنامه‌ریزی، ورودی ترمز، شمارندهٔ تکرار و توانایی تحریک DMA در چند حالت. در طراحی سخت‌افزاری برد ارزیابی CH32V203، سه گروه PWM مکملِ خروجیِ TIM1 فرمان‌های کنترل را اجرا می‌کنند.

**تایمر همگانی‌منظوره (GPTM)** در کنترل موتور نقش کمکی دارد. تایمر همگانی‌منظوره خروجی مکمل و درج زمان مرده ندارد و نمی‌تواند بازوهای بالا و پایین پل اینورتر سه‌فاز را مستقیماً برانگیزد؛ پس برای خروجی PWM مرحلهٔ توان، جایگزین تایمر پیشرفته نیست. معمولاً از آن برای گرفتن سیگنال‌ها استفاده می‌شود.

**تایمر ساده‌شده و تایمر پایه** کارکردهای تأخیر پایه و پالس کمکی را فراهم می‌کنند. سری CH32M007 یک تایمر 16 بیتیِ ساده‌شده دارد که در کنار تایمر پیشرفته، خروجی پالس با فرکانس معین یا کنترل تأخیر را انجام می‌دهد.

CH32V003 یک تایمر پیشرفته و یک تایمر همگانی‌منظوره دارد و از خروجی PWM مکمل و ورودی انکودر افزایشی پشتیبانی می‌کند؛ برای موتور جریان مستقیم جاروبک‌داری که تنها به یک کانال PWM برای تنظیم سرعت نیاز دارد مناسب است.

بخش ۲

# تایمرها در کاربرد موتور

## ◆ ۲.۱ پیکربندی حالت PWM

در کنترل موتور، تولید شکل‌موج PWM پایه‌ترین کارکرد تایمر است. تایمرهای WCH از دو حالت PWM پشتیبانی می‌کنند:

- **حالت PWM ۱:** وقتی مقدار شمارندهٔ مرکزی از مقدار رجیستر مقایسه/گرفتن کمتر باشد، OCxREF سطح بالا است و وقتی بیشتر شود سطح پایین می‌شود.
- **حالت PWM ۲:** برعکس حالت PWM ۱؛ وقتی مقدار شمارندهٔ مرکزی از مقدار رجیستر مقایسه/گرفتن کمتر باشد، OCxREF سطح پایین است.

بسته به حالت شمارش، دو نوع شکل‌موج PWM تولید می‌شود:

- **PWM هم‌ترازِ لبه:** شمارنده فقط صعودی یا فقط نزولی می‌شمارد؛ دوره توسط ATRLR و نسبت وظیفه توسط CHxCVR تعیین می‌شود.
- **PWM هم‌ترازِ مرکزی:** شمارنده به‌تناوب صعودی و نزولی می‌شمارد؛ دوره برابر دو برابرِ ATRLR و نسبت وظیفه برابر دو برابرِ CHxCVR است.

### ◆ ۲.۱.۱ پیکربندی PWM هم‌ترازِ لبه

کد زیر مقداردهی اولیهٔ نوعیِ تولید سه کانال PWM مکمل با TIM1 را نشان می‌دهد:

```
void TIM1_PWM_Init(u16 arr, u16 psc)
{
    TIM_TimeBaseInitTypeDef TIM_TimeBaseInitStructure = {0};
    TIM_OCInitTypeDef TIM_OCInitStructure = {0};
    GPIO_InitTypeDef GPIO_InitStructure = {0};

    /* Enable clocks */
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_TIM1, ENABLE);
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA | RCC_APB2Periph_GPIOB, ENABLE);

    /* Configure PWM output pins */
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_8 | GPIO_Pin_9 | GPIO_Pin_10;
```

```
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    /* Configure complementary output pins */
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_13 | GPIO_Pin_14 | GPIO_Pin_15;
    GPIO_Init(GPIOB, &GPIO_InitStructure);

    /* Time base configuration */
    TIM_TimeBaseInitStructure.TIM_Period = arr;
    TIM_TimeBaseInitStructure.TIM_Prescaler = psc;
    TIM_TimeBaseInitStructure.TIM_ClockDivision = TIM_CKD_DIV1;
    TIM_TimeBaseInitStructure.TIM_CounterMode = TIM_CounterMode_Up;
    TIM_TimeBaseInit(TIM1, &TIM_TimeBaseInitStructure);

    /* PWM channel configuration */
    TIM_OCInitStructure.TIM_OCMode = TIM_OCMode_PWM1;
    TIM_OCInitStructure.TIM_OutputState = TIM_OutputState_Enable;
    TIM_OCInitStructure.TIM_OutputNState = TIM_OutputNState_Enable;
    TIM_OCInitStructure.TIM_Pulse = arr / 2;  /* initial duty cycle 50% */
    TIM_OCInitStructure.TIM_OCPolarity = TIM_OCPolarity_High;
    TIM_OCInitStructure.TIM_OCNPolarity = TIM_OCNPolarity_High;
    TIM_OCInitStructure.TIM_OCIdleState = TIM_OCIdleState_Reset;
    TIM_OCInitStructure.TIM_OCNIdleState = TIM_OCNIdleState_Reset;
    TIM_OC1Init(TIM1, &TIM_OCInitStructure);
    TIM_OC2Init(TIM1, &TIM_OCInitStructure);
    TIM_OC3Init(TIM1, &TIM_OCInitStructure);

    /* Enable preload */
    TIM_OC1PreloadConfig(TIM1, TIM_OCPreload_Enable);
    TIM_OC2PreloadConfig(TIM1, TIM_OCPreload_Enable);
    TIM_OC3PreloadConfig(TIM1, TIM_OCPreload_Enable);
    TIM_ARRPreloadConfig(TIM1, ENABLE);
    TIM_CtrlPWMOutputs(TIM1, ENABLE);
    TIM_Cmd(TIM1, ENABLE);
}
```

نکات پیکربندی:

- **فعال‌سازی پیش‌بارگذاری:** با فعال‌کردن OCxPE و ARPE تضمین می‌شود پارامترها هم‌زمان با رویداد به‌روزرسانی اعمال شوند؛ پارامترهای PWM هنگام رویداد به‌روزرسانی هماهنگ به‌روز می‌شوند و از اعوجاج شکل‌موج جلوگیری می‌شود.
- **مقداردهی بیت UG:** پیش از راه‌اندازی شمارنده، بیت UG را برانگیزید تا همهٔ رجیسترها مقداردهی و رجیسترهای پیش‌بارگذاری مقدار اولیه بگیرند.
- **فعال‌سازی خروجی اصلی:** TIM_CtrlPWMOutputs برای فعال‌سازی خروجی اصلی تایمر پیشرفته (بیت MOE) به کار می‌رود؛ تایمر همگانی‌منظوره به این گام نیاز ندارد.

### ۲.۱.۲ پیکربندی PWM هم‌ترازِ مرکزی

کنترل FOC معمولاً حالت هم‌ترازِ مرکزی را برمی‌گزیند، چون شکل‌موج متقارن است و جای‌گذاری لحظهٔ نمونه‌برداری جریان را آسان می‌کند. در پیکربندی تنها کافی است حالت شمارش عوض شود:

```
TIM_TimeBaseInitStructure.TIM_CounterMode = TIM_CounterMode_CenterAligned1;
```

دورهٔ PWM هم‌ترازِ مرکزی برابر است با:

$$T_{PWM} = 2 * (ATRLR + 1) * (PSC + 1) / f_{CLK}$$

که در آن $f_{CLK}$ فرکانس ساعت تایمر است. پیش از راه‌اندازی شمارنده توصیه می‌شود بیت UG برانگیخته شود تا مقادیر پیش‌بارگذاری به‌درستی بارگذاری شوند.

### ۲.۱.۳ انتخاب مهندسی میان هم‌ترازی لبه و مرکز

در کنترل FOC، حالت PWM هم‌ترازِ مرکزی (CAPWM) نخستین انتخاب است. در این حالت شمارنده به‌تناوب صعودی و نزولی می‌شمارد و شکل‌موج PWM نسبت به مرکز دوره متقارن است؛ برای نمونه‌برداری جریان پنجرهٔ زمانی ثابتی فراهم می‌کند — هنگامی که شمارنده به قله می‌رسد، بازوهای پایینِ هر سه فاز هم‌زمان رسانا می‌شوند و جریان روی مقاومت نمونه‌برداری، جریان گذرگاه اصلی (باس) است؛ نسبت سیگنال به نویز بهترین حالت را دارد. دورهٔ CAPWM برابر دو برابرِ ATRLR و نسبت وظیفه برابر دو برابرِ CHxCVR است. پیش از راه‌اندازی شمارنده باید بیت UG را برانگیزید تا یک رویداد به‌روزرسانی نرم‌افزاری تولید شود و پیش از فعال‌سازی شمارنده، مقادیر رجیسترهای پیش‌بارگذاری به‌درستی بارگذاری شده باشند.

برای کنترل شش‌گامِ موج مربعیِ BLDC، حالت هم‌ترازِ لبه کفایت می‌کند. در این حالت شمارنده فقط صعودی می‌شمارد، دورهٔ PWM مستقیماً توسط ATRLR تعیین می‌شود و پیکربندی ساده‌تر است.

## ۲.۲ خروجی مکمل و پیکربندی زمان مرده

دو ماسفت بالایی و پایینیِ یک بازوی پل اینورتر سه‌فاز نمی‌توانند هم‌زمان رسانا شوند؛ در غیر این صورت اتصال مستقیم (شورت‌کردن گذرگاه) رخ می‌دهد. تایمر پیشرفته این مسئله را با خروجی مکمل و درج زمان مرده حل می‌کند.

زمان مرده از طریق میدان DTCFG در رجیستر BDTR پیکربندی می‌شود:

```
TIM_BDTRInitTypeDef TIM_BDTRInitStructure = {0};
TIM_BDTRInitStructure.TIM_OSSIState = TIM_OSSIState_Enable;
TIM_BDTRInitStructure.TIM_OSSRState = TIM_OSSRState_Enable;
TIM_BDTRInitStructure.TIM_LOCKLevel = TIM_LOCKLevel_1;
TIM_BDTRInitStructure.TIM_DeadTime = DEAD_TIME_VALUE;
TIM_BDTRInitStructure.TIM_Break = TIM_Break_Disable;
TIM_BDTRInitStructure.TIM_BreakPolarity = TIM_BreakPolarity_Low;
TIM_BDTRInitStructure.TIM_AutomaticOutput = TIM_AutomaticOutput_Enable;
TIM_BDTRConfig(TIM1, &TIM_BDTRInitStructure);
```

فرمول محاسبهٔ زمان مرده چنین است:

$$T_{DT} = DTCFG * T_{DTS}$$

که در آن $T_{DTS}$ گام زمان مرده است و به ساعت سیستم و پیکربندی تقسیم فرکانس بستگی دارد.

پیشنهادهایی برای گزینش زمان مرده:

- زمان مرده باید از زمان خاموش‌شدن ماسفت بیشتر باشد؛ معمولاً چند صد نانوثانیه تا چند میکروثانیه؛
- زمان مردهٔ بیش از اندازه به اعوجاج شکل‌موج خروجی و افت بازدهی موتور می‌انجامد؛

## ◆ ۲.۳ پیکربندی کارکرد ترمز

کارکرد ترمز سازوکار حفاظتی مهم در کنترل موتور است. هنگام رخ‌دادن اضافه‌جریان، اضافه‌ولتاژ یا خرابی دیگر، سیگنال ورودی ترمز می‌تواند بلافاصله خروجی تایمر را به وضعیت امن ببرد.

نکات پیکربندی ترمز:

```
/* Configure brake pin as pull-up input (for low-level trigger) */
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_12;
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IPU;  /* pull-up input */
GPIO_Init(GPIOB, &GPIO_InitStructure);

/* Brake function configuration */
TIM_BDTRInitStructure.TIM_Break = TIM_Break_Enable;
TIM_BDTRInitStructure.TIM_BreakPolarity = TIM_BreakPolarity_Low;
TIM_BDTRInitStructure.TIM_AutomaticOutput = TIM_AutomaticOutput_Disable;
```

**مشکل رایج:** هنگام پیکربندی ترمز، اگر حالت پایه درست تنظیم نشده باشد ممکن است «ترمز عمل نمی‌کند» یا «با وصل‌شدن برق، ترمز تحریک می‌شود» پیش بیاید. توصیه می‌شود پایهٔ ترمز به‌صورت ورودی با کشش بالا (برای تحریک با سطح پایین) پیکربندی شود تا پایه در حالت بی‌کارِ خود در سطح غیرمحرک باشد.

پس از تولید رویداد ترمز، بیت MOE صفر و بیت پرچم BIF یک می‌شود. اگر وقفه و DMA مربوطه فعال شده باشند، می‌توان وقفهٔ ترمز را برای پردازش برانگیخت.

## ◆ ۲.۴ سازوکار پیش‌بارگذاری

به‌روزرسانی پارامترهای PWM (نسبت وظیفه و دوره) باید با رجیسترهای پیش‌بارگذاری و به‌صورت بافر دوتایی انجام شود. در پیکربندی باید هم‌زمان OCxPE (پیش‌بارگذاری کانال) و ARPE (پیش‌بارگذاری بارگذاری مجدد خودکار) فعال شوند تا پارامترها هم‌زمان با رویداد به‌روزرسانی اعمال شوند و از پالس‌نویز (گلیچ) یا دورهٔ ناقص در لحظهٔ به‌روزرسانی شکل‌موج PWM جلوگیری شود.

## ◆ ۲.۵ تحریک ADC با تایمر

در کنترل FOC، نمونه‌برداری جریان باید با شکل‌موج PWM هم‌گام باشد تا در مدت رسانایی ماسفت‌ها جریان فاز به‌درستی گرفته شود. تایمر می‌تواند از طریق سیگنال TRGO (خروجی تحریک) نمونه‌برداری ADC را تحریک کند.

منبع ساعت تایمر پیشرفته می‌تواند به‌صورت سیگنال TRGO خروجی داده شود تا تایمرهای دیگر، ADC و جانبی‌های دیگر را تحریک کند.

```
/* Configure TIM1 TRGO to trigger ADC */
TIM_SelectOutputTrigger(TIM1, TIM_TRGOSource_Update);

/* ADC external trigger configuration */
ADC_InitStructure.ADC_ExternalTrigConv = ADC_ExternalTrigConv_T1_CC1;
ADC_InitStructure.ADC_ExternalTrigConvEdge = ADC_ExternalTrigConvEdge_Rising;
ADC_Init(ADC1, &ADC_InitStructure);

/* Enable ADC external trigger */
ADC_ExternalTrigConvCmd(ADC1, ENABLE);
```

**زمان نمونه‌برداری در حالت هم‌ترازِ مرکزی:** در PWM هم‌ترازِ مرکزی، شمارنده هنگام رسیدن به قله (مقدار ATRLR) رویداد به‌روزرسانی تولید می‌کند. در این لحظه بازوهای پایینِ هر سه فاز کاملاً رسانا هستند و جریان روی مقاومت نمونه‌برداری، جریان باس است؛ بهترین لحظه برای نمونه‌برداری جریان همین است. توصیه می‌شود تحریک ADC در قلهٔ شمارنده پیکربندی شود.

در طرح تک‌مقاومتی، باید بنا بر تشخیص سکتور SVPWM، در پنجرهٔ زمانی معینی نمونه‌برداری ADC تحریک شود. معمولاً از رویداد مقایسه (نه رویداد به‌روزرسانی) برای تحریک ADC استفاده می‌شود تا لحظهٔ نمونه‌برداری به‌دقت کنترل شود.

## ◆ ۲.۶ پیکربندی انکودر

تایمر پیشرفته و تایمر همگانی‌منظوره هر دو از حالت انکودر افزایشی پشتیبانی می‌کنند. حالت انکودر کاربرد نمونه‌ای تایمر است و می‌توان خروجی دو فاز انکودر را به آن داد؛ جهت شمارش شمارندهٔ مرکزی با جهت محور انکودر هم‌گام است و هر پالس خروجی انکودر، شمارندهٔ مرکزی را یکی زیاد یا یکی کم می‌کند.

برای فعال‌سازی حالت انکودر کافی است میدان SMS روی 001b (شمارش فقط با لبه‌های TI2)، 010b (شمارش فقط با لبه‌های TI1) یا 011b (شمارش با لبه‌های TI1 و TI2) تنظیم شود.

```
void Encoder_Init(void)
{
    TIM_TimeBaseInitTypeDef TIM_TimeBaseInitStructure = {0};
    TIM_ICInitTypeDef TIM_ICInitStructure = {0};
    GPIO_InitTypeDef GPIO_InitStructure = {0};

    /* Enable clocks */
    RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM3, ENABLE);
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE);

    /* Configure encoder A/B phase input pins */
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_6 | GPIO_Pin_7;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN_FLOATING;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    /* Time base configuration */
    TIM_TimeBaseInitStructure.TIM_Period = 65535;
    TIM_TimeBaseInitStructure.TIM_Prescaler = 0;
    TIM_TimeBaseInitStructure.TIM_ClockDivision = TIM_CKD_DIV1;
    TIM_TimeBaseInitStructure.TIM_CounterMode = TIM_CounterMode_Up;
    TIM_TimeBaseInit(TIM3, &TIM_TimeBaseInitStructure);

    /* Encoder interface configuration */
    TIM_EncoderInterfaceConfig(TIM3, TIM_EncoderMode_TI12, TIM_ICPolarity_Rising,
    TIM_ICPolarity_Rising);

    /* Input capture filter configuration */
    TIM_ICInitStructure.TIM_Channel = TIM_Channel_1;
    TIM_ICInitStructure.TIM_ICFilter = 0x0F;
    TIM_ICInit(TIM3, &TIM_ICInitStructure);
    TIM_ICInitStructure.TIM_Channel = TIM_Channel_2;
    TIM_ICInit(TIM3, &TIM_ICInitStructure);
    TIM_Cmd(TIM3, ENABLE);
}
```

**پیکربندی مضارب:** با انتخاب حالت انکودر می‌توان مضارب مختلفی به دست آورد:

- شمارش فقط با لبه‌های TI1: مضرب ۱؛
- شمارش فقط با لبه‌های TI2: مضرب ۱؛
- شمارش با لبه‌های TI1 و TI2: مضرب ۲؛

**تشخیص جهت:** جهت چرخش انکودر با خواندن بیت DIR در رجیستر TIMx->CR1 تعیین می‌شود.

در کنترل FOC موتور PMSM، انکودر برای به‌دست‌آوردن موقعیت و سرعت روتور به کار می‌رود. گام‌های پیکربندی عبارت‌اند از:

- مقداردهی تایمر رابط انکودر و تعیین دامنهٔ شمارش مناسب؛
- خواندن مقدار شمارنده در هر دورهٔ کنترل و محاسبهٔ افزایش موقعیت؛
- محاسبهٔ سرعت چرخش از روی افزایش موقعیت؛
- ترکیب اطلاعات موقعیت برای تبدیل Park و تبدیل معکوس Park.

> **توجه:** هنگام سرریز شمارنده باید به‌درستی رفتار شود. توصیه می‌شود تعداد سرریزها در وقفه ثبت شود و موقعیت کل از ترکیب مقدار شمارنده محاسبه گردد.

## ◆ ۲.۷ تحریک DMA با TIM

تایمر از DMA در حالت‌های متعدد پشتیبانی می‌کند و می‌تواند با رویدادهای زیر درخواست DMA بدهد:

- رویداد به‌روزرسانی (UEV)
- رویداد مقایسه/گرفتن (CCx)
- رویداد تحریک (TRG)

DMA می‌تواند مقادیر رجیستر مقایسهٔ تایمر را خودکار به‌روز کند و بار CPU را سبک‌تر کند. کاربردهای نمونهٔ DMA در کنترل موتور عبارت‌اند از:

- **به‌روزرسانی گروهی نسبت وظیفهٔ PWM:** به‌روزرسانی خودکار مقدار نسبت وظیفه در چند دورهٔ PWM و پیاده‌سازی توالی شکل‌موج‌های از پیش تعریف‌شده.
- **انتقال داده‌های نمونه‌برداری ADC:** جابه‌جایی خودکار نتایج تبدیل ADC به ناحیهٔ مشخصی از حافظه برای مصرف الگوریتم FOC.

```
/* Configure TIM1 update event to trigger DMA */
TIM_DMACmd(TIM1, TIM_DMA_Update, ENABLE);

/* DMA configuration */
DMA_InitStructure.DMA_PeripheralBaseAddr = (u32)&TIM1->CH1CVR;
DMA_InitStructure.DMA_MemoryBaseAddr = (u32)pwm_buffer;
DMA_InitStructure.DMA_DIR = DMA_DIR_PeripheralDST;
DMA_InitStructure.DMA_BufferSize = BUFFER_SIZE;
DMA_InitStructure.DMA_PeripheralInc = DMA_PeripheralInc_Disable;
DMA_InitStructure.DMA_MemoryInc = DMA_MemoryInc_Enable;
DMA_InitStructure.DMA_PeripheralDataSize = DMA_PeripheralDataSize_HalfWord;
DMA_InitStructure.DMA_MemoryDataSize = DMA_MemoryDataSize_HalfWord;
DMA_InitStructure.DMA_Mode = DMA_Mode_Circular;
DMA_InitStructure.DMA_Priority = DMA_Priority_High;
DMA_Init(DMA1_Channel1, &DMA_InitStructure);
DMA_Cmd(DMA1_Channel1, ENABLE);
```

کنترل‌گر DMA از مدیریت بافر حلقوی پشتیبانی می‌کند. در کنترل موتور، حالت حلقه به‌ویژه برای انتقال پیوستهٔ داده‌های نمونه‌برداری ADC مناسب است؛ به‌روزرسانی خودکار داده‌های نمونه بدون دخالت CPU حاصل می‌شود.

## ◆ ۲.۸ مدیریت و بهینه‌سازی وقفه‌های تایمر

میکروکنترلر به‌طور پیش‌فرض تو در تویی وقفه‌ها را می‌بندد و باید اولویت و فعال‌سازی وقفه‌ها به‌صورت دستی از طریق PLIC پیکربندی شود. در کنترل موتور، پیکربندی درست اولویت وقفه‌ها اهمیتی حیاتی دارد:

- **بالاترین اولویت:** وقفهٔ ترمز (TIM1_BRK_IRQHandler) — تضمین پاسخ فوری هنگام خرابی
- **اولویت بالا:** وقفهٔ پایان نمونه‌برداری ADC — برای محاسبهٔ حلقه جریان
- **اولویت میانی:** وقفهٔ به‌روزرسانی تایمر — برای محاسبهٔ حلقه سرعت و به‌روزرسانی PWM
- **اولویت پایین:** وقفهٔ گرفتن انکودر — برای به‌روزرسانی موقعیت

```
/* Configure interrupt priorities */
NVIC_SetPriority(TIM1_BRK_IRQn, 0);      /* brake interrupt: highest priority */
NVIC_SetPriority(ADC_IRQn, 1);           /* ADC interrupt: high priority */
NVIC_SetPriority(TIM1_UP_IRQn, 2);       /* timer update interrupt */
NVIC_SetPriority(TIM3_IRQn, 3);          /* encoder interrupt */

NVIC_EnableIRQ(TIM1_BRK_IRQn);
NVIC_EnableIRQ(ADC1_2_IRQn);
NVIC_EnableIRQ(TIM1_UP_IRQn);
NVIC_EnableIRQ(TIM3_IRQn);
```

برای بالا بردن پاسخ‌گویی لحظه‌ای کنترل موتور، برنامه سرویس وقفه باید تا جای ممکن کوتاه باشد:

- **وقفهٔ ADC:** تنها نمونه‌برداری جریان و تبدیل مختصات انجام می‌شود؛ بخش اصلی الگوریتم FOC می‌تواند همین‌جا کامل شود.
- **وقفهٔ به‌روزرسانی تایمر:** برای محاسبهٔ حلقه سرعت؛ فرکانس آن معمولاً 1/5~1/10 فرکانس حلقه جریان است.
- **وقفهٔ ترمز:** بلافاصله خروجی PWM بسته و وضعیت خرابی ثبت می‌شود.

```
void TIM1_BRK_IRQHandler(void)
{
    if (TIM_GetITStatus(TIM1, TIM_IT_Break) != RESET)
    {
        /* Turn off PWM output immediately */
        TIM_CtrlPWMOutputs(TIM1, DISABLE);
        /* Record fault status */
        motor_state.fault_flag = FAULT_BREAK;
        /* Clear interrupt flag */
        TIM_ClearITPendingBit(TIM1, TIM_IT_Break);
    }
}
```

پیوست

# تاریخچهٔ نسخه‌ها و بیانیه

جدول تاریخچه تاریخچهٔ نسخه‌های سند اصلی AN11002

| تاریخ | نسخه | شرح تغییرات |
|---|---|---|
| 2026/07/08 | V1.0 | انتشار نسخهٔ اول |

## بیانیه

این راهنما صرفاً جهت مرجع ارائه شده است و نویسندهٔ سند اصلی می‌تواند بدون اطلاع قبلی و در هر زمان، تغییراتی در محتوای آن اعمال کند. ترجمهٔ فارسی این سند توسط تیم معمار الکترونیک صنعت تهیه شده است؛ برای اطمینان از به‌روزترین نسخه به https://wch.cn مراجعه کنید.

پیوست

# واژگان کلیدی (واژه‌نامهٔ فنی)

اصطلاحات فنی سخت‌افزار مطابق سند اصلی به‌صورت انگلیسی حفظ شده‌اند. جدول زیر معادل‌ها و شرح کوتاه هر واژه را برای مرجع سریع جمع می‌کند.

جدول واژگان واژگان کلیدی به‌کاررفته در این جزوه

| واژه | یعنی چه؟ |
|---|---|
| ADTM | تایمر پیشرفته؛ با خروجی مکمل، درج زمان مرده، ورودی ترمز، شمارندهٔ تکرار و تحریک DMA در چند حالت؛ هستهٔ اصلی راندن مرحلهٔ توان در کنترل موتور |

| واژه | یعنی چه؟ |
|---|---|
| GPTM | تایمر همگانی‌منظوره؛ فاقد خروجی مکمل و زمان مرده، معمولاً برای گرفتن سیگنال انکودر یا هال و کارهای کمکی به کار می‌رود |
| FOC | کنترل میدان‌جهت‌یافته؛ روش کنترل برداری موتور با تبدیل‌های مختصات و حلقه‌های جریان و سرعت |
| PWM | مدولاسیون پهنای پالس؛ کنترل ولتاژ مؤثر موتور از راه تغییر نسبت روشن/خاموش بودن سیگنال |
| BLDC | موتور جریان مستقیم بدون جاروبک؛ با موج مربعیِ شش‌گام یا FOC رانده می‌شود |
| PMSM | موتور سنکرون با مغناطیس دائم؛ معمولاً با FOC و PWM هم‌ترازِ مرکزی کنترل می‌شود |
| Dead-time | زمان مرده؛ فاصلهٔ زمانی تحمیلی میان خاموش‌شدن یک ترانزیستور و روشن‌شدن مکمل آن برای جلوگیری از اتصال مستقیم بازوهای بالا و پایین |
| Complementary output | خروجی مکمل؛ جفت خروجی CHx/CHxN با منطق وارونه و درج زمان مرده در تایمر پیشرفته |
| Brake | کارکرد ترمز؛ ورودی حفاظتی که در خرابی، خروجی تایمر را بی‌درنگ به وضعیت امن می‌برد و بیت MOE را صفر می‌کند |
| Encoder | انکودر افزایشی؛ حسگر دو فاز A/B که با حالت انکودر تایمر برای تعیین موقعیت و سرعت روتور به کار می‌رود |
| ATRLR / CHxCVR | رجیستر بارگذاری مجدد خودکار (تعیین‌کنندهٔ دورهٔ PWM) / رجیستر مقایسه‌گر کانال (تعیین‌کنندهٔ نسبت وظیفه) |
| TRGO | خروجی تحریک تایمر؛ سیگنالی برای هم‌گام‌سازی نمونه‌برداری ADC یا تحریک تایمرهای دیگر |
| DMA | دسترسی مستقیم به حافظه؛ انتقال خودکار داده‌های نمونه یا به‌روزرسانی مقایسه‌گرها بدون بار روی CPU |
| MOE / BIF | بیت فعال‌سازی خروجی اصلی / پرچم رویداد ترمز در رجیستر BDTR؛ وضعیت پس از رویداد ترمز را نشان می‌دهند |




# معمار الکترونیک صنعت

MEMAR ELECTRONIC SANAT (MES)

در مورد مطالب این جزوه و ترجمه، بدیهی است که از نقطه‌نظرات و پیشنهادهای شما عزیزان در این راستا خوشحال می‌شویم. همچنین اشکالات، کم‌وکاستی‌ها و پرسش‌های خود را در این مورد با ما در میان بگذارید تا در نسخه‌های بعدی لحاظ شوند.

**با تشکر، حسن معماری**

ترجمهٔ فارسی سند AN11002 V1.0 · نسخهٔ ۱٫۰ — تاریخ ساخت: ۱۴۰۵/۰۸/۱۹

## راه‌های ارتباط با ما

| | |
|---|---|
| گیت‌هاب | https://github.com/MEMAR-ELECTRONIC-SANAT |
| کانال تلگرام | https://t.me/Memar_Electronic |
| پیج اینستاگرام | https://www.instagram.com/Memar.Electronic |
| ایمیل | Memari.h@gmail.com |

آموزش، طراحی و ساخت بردهای الکترونیکی صنعتی — این جزوه با افتخار توسط تیم معمار الکترونیک صنعت برای جامعهٔ مهندسی ایران تهیه شده است.

ترجمهٔ فارسی بر پایهٔ سند رسمی AN11002 نسخهٔ V1.0 منتشرهٔ WCH — https://wch.cn